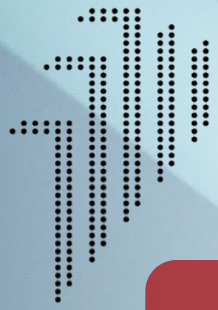
h_da
hochschule
darmstadt

member of
EUT+
EUROPEAN UNIVERSITY OF TECHNOLOGY



h_da

HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

Natural Language Processing – Text classification

Introduction to Artificial Intelligence

Prof. Dr. rer. nat. Hamed Shariat Yazdi

Department of Computer Science

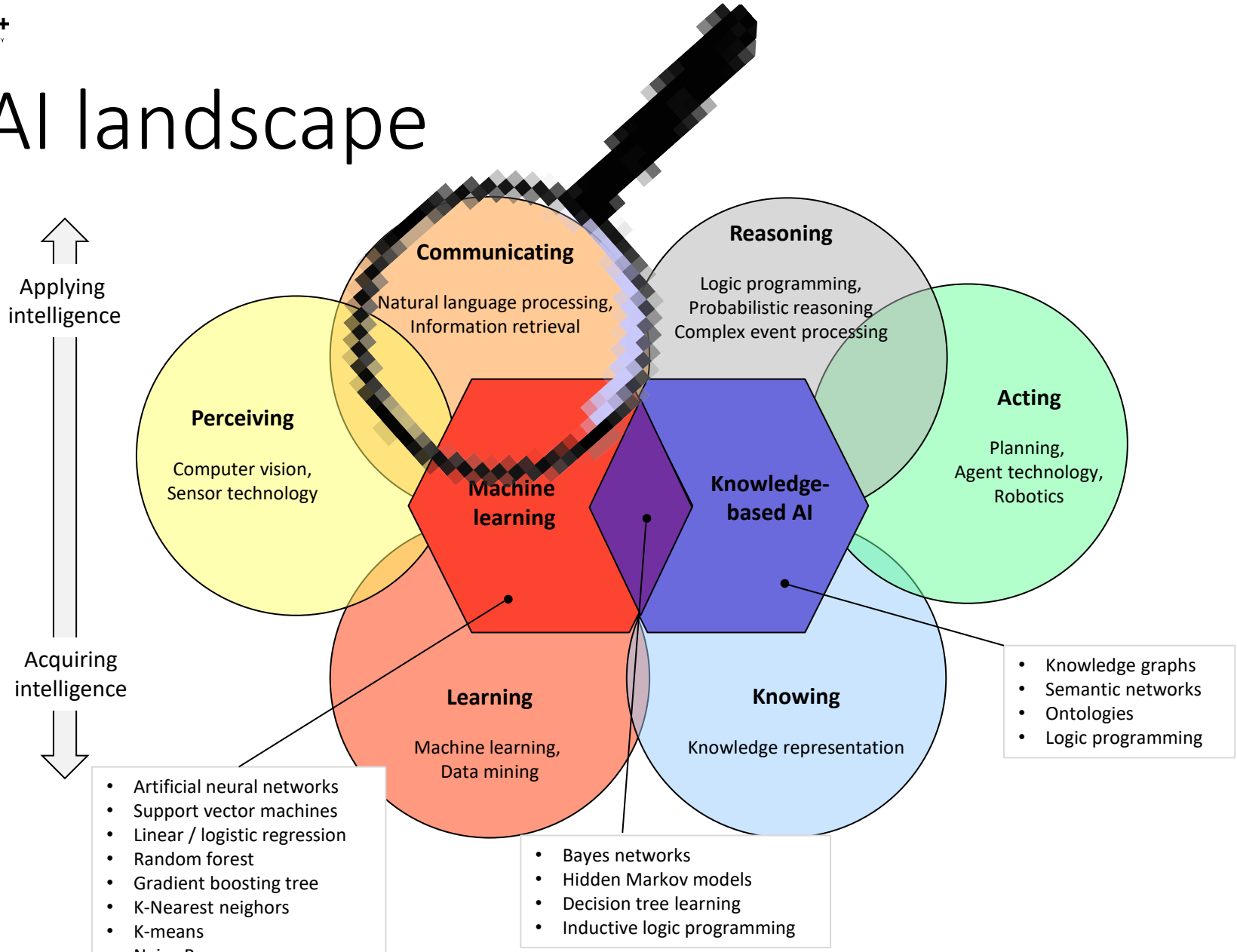
Darmstadt University of Applied Sciences

Agenda

- Overview
- Bag-of-words model
- Example
- Quick check

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

AI landscape



Inspired by https://www.sigs-datacom.de/order/poster/Landkarte-KI-Poster-AI-2018.php

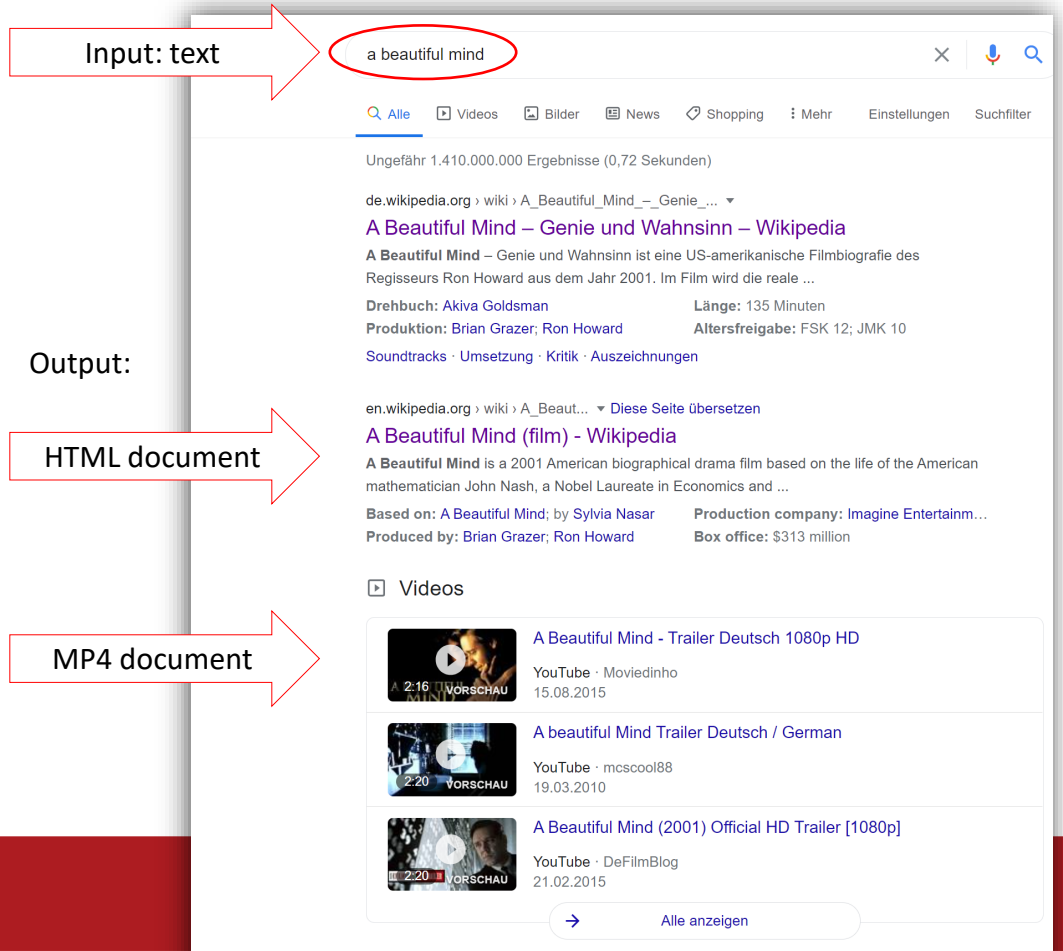
Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Natural Language Processing (NLP) Areas

Information Retrieval (IR)	• Retrieving relevant documents (“Document retrieval“ would be more suitable a term)
Text Classification	• Classifying natural language texts, e.g. to detect languages, topics, or sentiment
Information Extraction (IE)	• Understanding written / spoken texts • Building knowledge graphs from texts
Question Answering (QA)	• Answering natural language questions
Machine Translation	• Translating texts between natural languages
Text Generation	• Generate written / spoken texts • Summarizing texts

Information retrieval (= document retrieval)

- Input: text → Output: relevant documents



Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Text classification
Example: Language detection

- Input: text → Output: classification

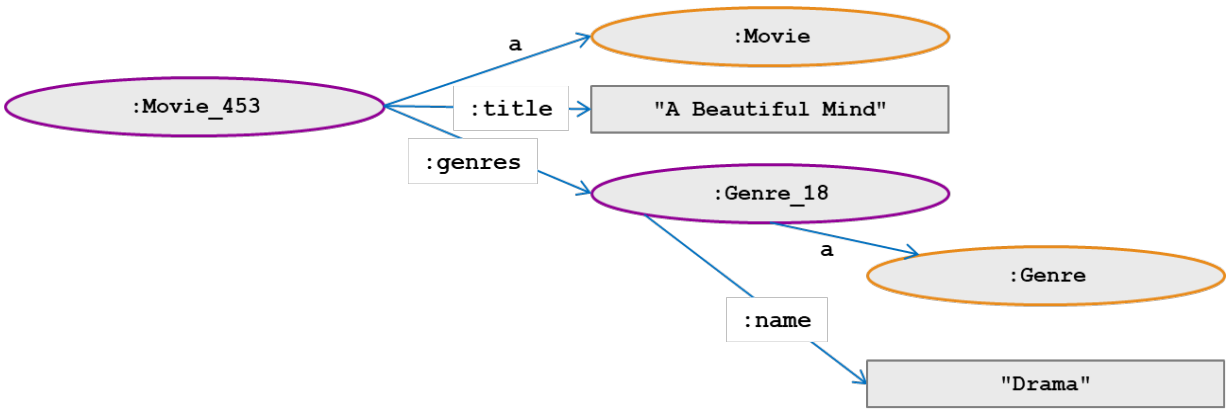
Text	Classification
A Beautiful Mind is a 2001 American biographical drama film based on the life of the American mathematician John Nash, a Nobel Laureate in Economics and Abel Prize winner. The film was directed by Ron Howard, from a screenplay written by Akiva Goldsman.	→ EN
A Beautiful Mind ist ein amerikanischer biografischer Drama-Film aus dem Jahr 2001, der auf dem Leben des amerikanischen Mathematikers John Nash, einem Nobelpreisträger für Wirtschaftswissenschaften und Abel-Preisträger, basiert. Regie führte Ron Howard, nach einem Drehbuch von Akiva Goldsman.	→ DE

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Information extraction

- Input: text → Output: knowledge graph

"A Beautiful Mind" is a movie which belongs to the genre drama.



Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Question answering

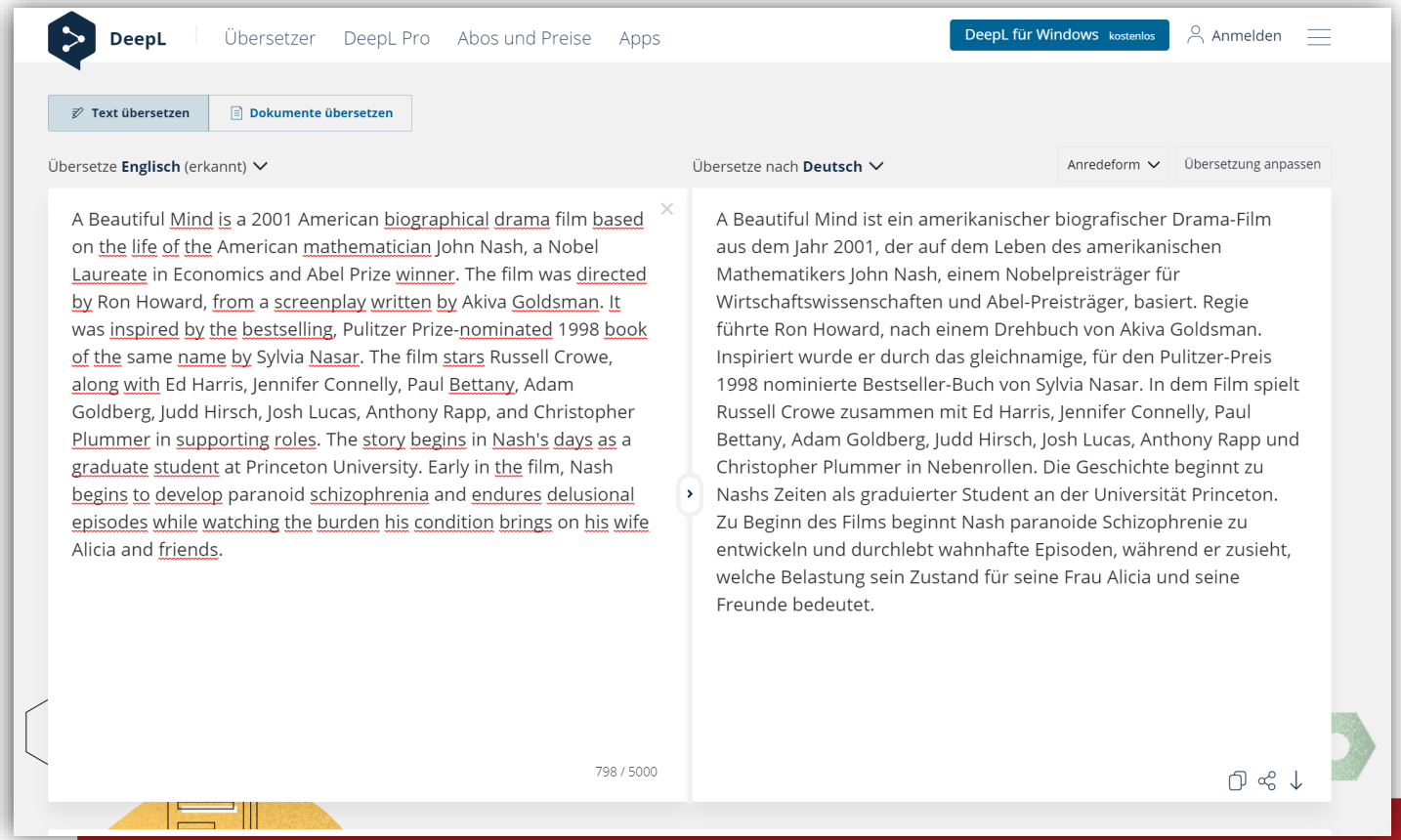
- Input: question text → Output: answer text



https://upload.wikimedia.org/wikipedia/commons/5/5c/Amazon_Echo.jpg

Machine translation

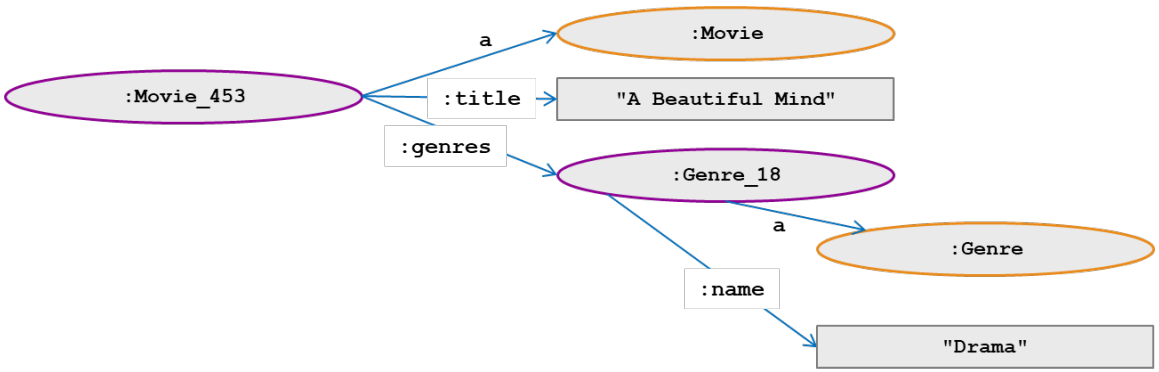
- Input: text in one language → Output: text in another language



https://www.deepl.com

Text generation

- Input: knowledge graph → Output: text



„A Beautiful Mind“ is a movie which belongs to the genre drama.

Agenda

- Overview
- Bag-of-words model
- Example
- Quick check

Bag-of-words model

- Simple NLP approach
- Surprisingly good results in certain application scenarios, e.g., text classification
- Idea:
  Text is represented as the bag (multiset) of its words, disregarding grammar and even word order but only keeping multiplicity
- Example:

```
"John likes to watch movies. Mary likes movies too."

BoW ={"John":1,"likes":2,"to":1,"watch":1,"movies":2,
"Mary":1,"too":1};
```

https://en.wikipedia.org/wiki/Bag-of-words_model

Application

- Use Bag-of-words model for **generating feature vectors** for machine learning
- Simple form: **term frequency**
  Each entry in the vector denotes the number of appearances of a term in the text.
- Example text t1:

ID	John	likes	to	watch	movies	Mary	too	…	label
t1	1	2	1	1	2	1	1	…	A
t2	0	0	2	0	0	0	1	…	C
…	…	…	…	…	…	…	…	…	…
tn	0	1	2	2	3	0	0	…	A

- “John“ appears 1 time in the text, “ likes“ 2 times, and so forth

tf-idf (term frequency – inverse document frequency

- Terms with a high frequency are not necessarily most important.
  Examples: stop words like "the", "a", "to"
- → Normalization of term frequency by inverse document frequency
- **Term frequency**: frequency of term in one text
- **Document frequency**: frequency of term in all texts (entire corpus)
- Various formulas for tf-idf in use,
  see https://en.wikipedia.org/wiki/Tf%E2%80%93idf

N-gram model

- Simple bag-of-words model considers individual terms only
- Extension: consider sequences of N terms
- Example: N=2

1-gram 2-gram

ID	John	likes	to	…	John likes	likes to	to watch	…	label
t1	1	2	1	…	1	1	1	…	A
t2	0	0	2	…	0	0	1	…	C
…	…	…	…	…	…	…	…	…	…
tn	0	1	2	…	0	1	2	…	A

- N-gram model can be used for generating feature vectors, too
- Also with tf-idf

https://en.wikipedia.org/wiki/Bag-of-words_model

Discussion bag-of-words model

- What are advantages and disavantages of the bag-of-words model?



https://cdn.pixabay.com/photo/2015/11/03/08/56/question-mark-1019820_960_720.jpg

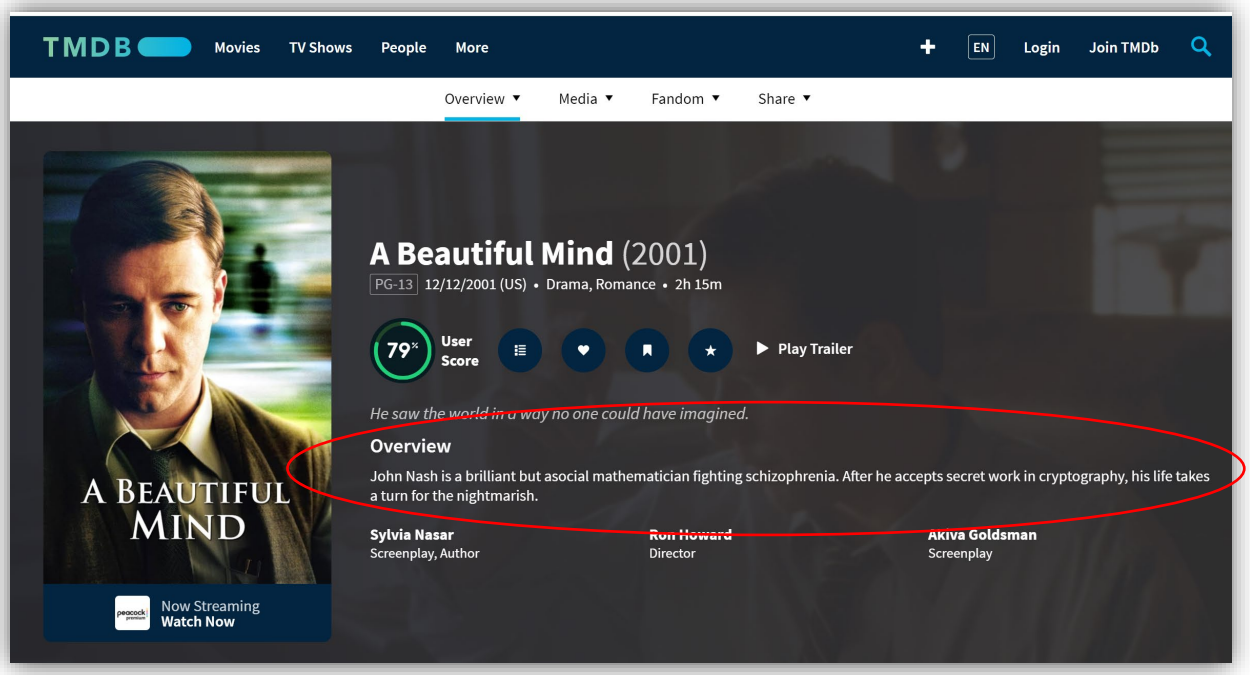
Agenda

- Overview
- Bag-of-words model
- Example
- Quick check

Example: Predicting genre from movie overview

Input: movie overview text

Output: genre category



Drama

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

https://www.themoviedb.org/movie/453

Training data: Movie_Overview_Classification.csv

Input: overview

Output: binary classification Drama

	id	overview	genre_Drama
2	1	When Lou, who has become the "father of the Internet," is shot by an unknown assailant, Jacob and Nick fire up the time mac	0
3	2	Mia Thermopolis is now a college graduate and on her way to Genovia to take up her duties as princess. Her best friend Lilly	1
4	3	Under the direction of a ruthless instructor, a talented young drummer begins to pursue perfection at any cost, even his huma	1
5	4	Vidya Bagchi (Vidya Balan) arrives in Kolkata from London to find her missing husband Arnab Bagchi. Seven months pregnan	1
6	5	Marine Boy is the story of a former national swimmer who finds himself in debt, and out of desperation is employed as a mule	0
7	6	Pinocchio and his friends, a glow worm and a marionette, search for a magic music box. However, so are the evil Scalawag a	0
8	7	A young girl buys an antique box at a yard sale, unaware that inside the collectible lives a malicious ancient spirit. The girl's fa	0
9	8	A chronicle which provides a rare window into the international perception of the Iraq War, courtesy of Al Jazeera, the Arab wo	0
10	9	After telling the story of Flint's last journey to young Jim Hawkins, Billy Bones has a heart attack and dies just as Jim and his fr	0
11	10	In "A Mighty Wind", director Christopher Guest reunites the team from "Best In Show" and "Waiting for Guffman" to tell tell the	0
12	11	When world heavyweight boxing champion, Apollo Creed wants to give an unknown fighter a shot at the title as a publicity stu	1
13	12	The members of the Lambda Lambda Lambda fraternity travel to Fort Lauderdale for a fraternity conference. They'll have to b	0
14	13	Lester Burnham, a depressed suburban father in a mid-life crisis, decides to turn his hectic life around after developing an infa	1
15	14	Disenchanted with the movie industry, Chili Palmer tries the music industry, meeting and romancing a widow of a music exec	0
16	15	John Anderton is a top 'Precrime' cop in the late-21st century, when technology can predict crimes before they're committed. I	0
17	16	Novica is a mathematics champion in a Belgrade high school. In an attempt to overcome the ‘geek’ status at school, Novica b	1
18	17	After attending the funeral of her grandmother in Dallas, the Lux Atlantic Hotel manager Lisa is waiting for a flight to Miami. Du	0
19	18	In 1857, at the height of his fame and fortune, novelist and social critic Charles Dickens meets and falls in love with teenage s	1
20	19	While working a job at an exclusive ski resort to support her Dad, Kim learns to snowboard and is so good at it that she enters	0
21	20	Professional driver Frank Martin is living in Miami, where he is temporarily filling in for a friend as the chauffeur for a governme	0
22	21	The prospects for continuing life on Earth in the year 2058 are grim. So the Robinsons are launched into space to colonize Alp	0
23	22	An idyllic summer turns into a nightmare of unspeakable terror for yet another group of naive counselors. Ignoring Camp Crys	0
24	23	When a group of misfits is hired by an unknown third party to burglarize a desolate house and acquire a rare VHS tape, they c	0
25	24	Kaitlyn, an ER nurse who is tending to a young stabbing victim, is accidentally electrocuted by the defibrillator that is used in a	0
26	25	A genetic engineering experiment gone horribly awry turns a large flock of docile sheep into unrelenting killing machines in thi	0
27	26	In 1957, Evan Rendell flees after his father is lynched for killing multiple patients in his effort to find a replacement heart for his	0
28	27	An employee of a corporation with a lucrative secret process is tempted to betray it. But there's more to it than that.	1

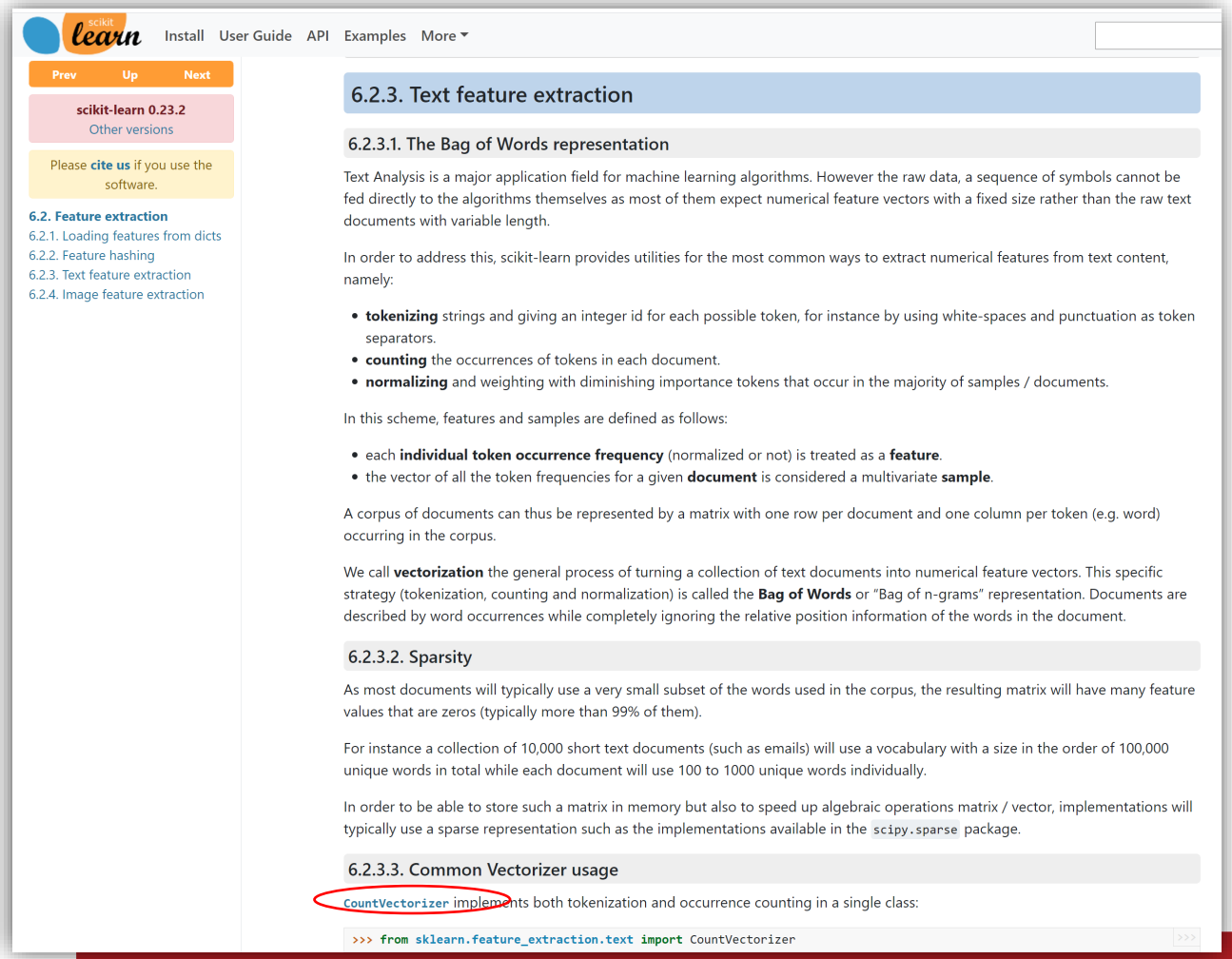
Loading DataFrame from CSV file

```
import pandas as pd
```

```
data = pd.read_csv('data/Movie_Overview_Classification.csv',
                   encoding='utf8',
                   delimiter=',',
                   quotechar='|',
                   quoting=csv.QUOTE_MINIMAL)
```

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html

Bag-of-words model with Scikit-learn



scikit learn Install User Guide API Examples More

Prev Up Next

scikit-learn 0.23.2
Other versions

Please cite us if you use the software.

6.2. Feature extraction
6.2.1. Loading features from dicts
6.2.2. Feature hashing
6.2.3. Text feature extraction
6.2.4. Image feature extraction

6.2.3. Text feature extraction

6.2.3.1. The Bag of Words representation

Text Analysis is a major application field for machine learning algorithms. However the raw data, a sequence of symbols cannot be fed directly to the algorithms themselves as most of them expect numerical feature vectors with a fixed size rather than the raw text documents with variable length.

In order to address this, scikit-learn provides utilities for the most common ways to extract numerical features from text content, namely:

- **tokenizing** strings and giving an integer id for each possible token, for instance by using white-spaces and punctuation as token separators.
- **counting** the occurrences of tokens in each document.
- **normalizing** and weighting with diminishing importance tokens that occur in the majority of samples / documents.

In this scheme, features and samples are defined as follows:

- each **individual token occurrence frequency** (normalized or not) is treated as a **feature**.
- the vector of all the token frequencies for a given **document** is considered a multivariate **sample**.

A corpus of documents can thus be represented by a matrix with one row per document and one column per token (e.g. word) occurring in the corpus.

We call **vectorization** the general process of turning a collection of text documents into numerical feature vectors. This specific strategy (tokenization, counting and normalization) is called the **Bag of Words** or "Bag of n-grams" representation. Documents are described by word occurrences while completely ignoring the relative position information of the words in the document.

6.2.3.2. Sparsity

As most documents will typically use a very small subset of the words used in the corpus, the resulting matrix will have many feature values that are zeros (typically more than 99% of them).

For instance a collection of 10,000 short text documents (such as emails) will use a vocabulary with a size in the order of 100,000 unique words in total while each document will use 100 to 1000 unique words individually.

In order to be able to store such a matrix in memory but also to speed up algebraic operations matrix / vector, implementations will typically use a sparse representation such as the implementations available in the `scipy.sparse` package.

6.2.3.3. Common Vectorizer usage

`CountVectorizer` implements both tokenization and occurrence counting in a single class:

```
>>> from sklearn.feature_extraction.text import CountVectorizer
```

https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Tokenizing: CountVectorizer

Tokenizing text with `scikit-learn`

Text preprocessing, tokenizing and filtering of stopwords are all included in **`CountVectorizer`**, which builds a dictionary of features and transforms documents to feature vectors:

```
>>> from sklearn.feature_extraction.text import CountVectorizer
>>> count_vect = CountVectorizer()
>>> X_train_counts = count_vect.fit_transform(twenty_train.data)
>>> X_train_counts.shape
(2257, 35788)
```

`CountVectorizer` supports counts of N-grams of words or consecutive characters. Once fitted, the vectorizer has built a dictionary of feature indices:

```
>>> count_vect.vocabulary_.get(u'algorithm')
4690
```

The index value of a word in the vocabulary is linked to its frequency in the whole training corpus.

https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html

tf-idf transformation: TfidfTransformer

From occurrences to frequencies

Occurrence count is a good start but there is an issue: longer documents will have higher average count values than shorter documents, even though they might talk about the same topics.

To avoid these potential discrepancies it suffices to divide the number of occurrences of each word in a document by the total number of words in the document: these new features are called `tf` for Term Frequencies.

Another refinement on top of tf is to downscale weights for words that occur in many documents in the corpus and are therefore less informative than those that occur only in a smaller portion of the corpus.

This downscaling is called tf–idf for "Term Frequency times Inverse Document Frequency".

Both **tf** and **tf–idf** can be computed as follows using `TfidfTransformer`:

```
>>> from sklearn.feature_extraction.text import TfidfTransformer
>>> tf_transformer = TfidfTransformer(use_idf=False).fit(X_train_counts)
>>> X_train_tf = tf_transformer.transform(X_train_counts)
>>> X_train_tf.shape
(2257, 35788)
```

In the above example-code, we firstly use the `fit(..)` method to fit our estimator to the data and secondly the `transform(..)` method to transform our count-matrix to a tf-idf representation. These two steps can be combined to achieve the same end result faster by skipping redundant processing. This is done through using the `fit_transform(..)` method as shown below, and as mentioned in the note in the previous section:

```
>>> tfidf_transformer = TfidfTransformer()
>>> X_train_tfidf = tfidf_transformer.fit_transform(X_train_counts)
>>> X_train_tfidf.shape
(2257, 35788)
```

https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html

Training: MultinomialNB

Training a classifier

Now that we have our features, we can train a classifier to try to predict the category of a post. Let's start with a naïve Bayes classifier, which provides a nice baseline for this task. `scikit-learn` includes several variants of this classifier; the one most suitable for word counts is the multinomial variant:

```
>>> from sklearn.naive_bayes import MultinomialNB
>>> clf = MultinomialNB().fit(X_train_tfidf, twenty_train.target)
```

To try to predict the outcome on a new document we need to extract the features using almost the same feature extracting chain as before. The difference is that we call `transform` instead of `fit_transform` on the transformers, since they have already been fit to the training set:

```
>>> docs_new = ['God is love', 'OpenGL on the GPU is fast']
>>> X_new_counts = count_vect.transform(docs_new)
>>> X_new_tfidf = tfidf_transformer.transform(X_new_counts)

>>> predicted = clf.predict(X_new_tfidf)

>>> for doc, category in zip(docs_new, predicted):
...     print('%r => %s' % (doc, twenty_train.target_names[category]))
...
'God is love' => soc.religion.christian
'OpenGL on the GPU is fast' => comp.graphics
```

Configuring a compound classifier: Pipeline

Building a pipeline

In order to make the vectorizer => transformer => classifier easier to work with, `scikit-learn` provides a `Pipeline` class that behaves like a compound classifier:

```
>>> from sklearn.pipeline import Pipeline
>>> text_clf = Pipeline([
...     ('vect', CountVectorizer()),
...     ('tfidf', TfidfTransformer()),
...     ('clf', MultinomialNB()),
... ])
```

The names `vect`, `tfidf` and `clf` (classifier) are arbitrary. We will use them to perform grid search for suitable hyperparameters below. We can now train the model with a single command:

```
>>> text_clf.fit(twenty_train.data, twenty_train.target)
Pipeline(...)
```

https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Cross-validation: Accuracy

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y)
print(f'Accuracy: {scores.mean()}')
```

Agenda

- Overview
- Bag-of-words model
- Example
- Quick check

Quick check
NLP – text classification

1. Name and explain different areas of NLP
2. What is the task of text classification?
3. Explain the bag-of-words model
4. What are advantages and disadvantages of the bag-of-words model?
5. Explain tf-idf
6. Explain the n-gram model
7. How to use Pipeline in scikit-learn?